

به نام خدا

## سیستم های مدیریت هوشمند انرژی ساختمان ها

هستی قمشی ، تانیا اسدی شه میرزادی، ریحانه استادغلامی

### فاز ۶: پیاده سازی نمونه اولیه

#### مقدمه

مدیریت بهینه انرژی در ساختمان های هوشمند، مستلزم سیستمی است که بتواند با دقت بالا، داده های محیطی را پایش کرده و بر اساس منطقی قابل اعتماد، واکنش نشان دهد. پروژه Smart Energy System با هدف ارائه راهکاری یکپارچه برای این منظور توسعه یافته است. در طراحی این سیستم، اولویت اصلی بر اصول شیء گرایی و ماژولاریتی متمرکز بوده است تا خروجی نهایی نه تنها از نظر عملیاتی کارآمد، بلکه از نظر ساختار نرم افزاری کاملاً استاندارد و توسعه پذیر باشد.

#### شرح عملکرد نمونه ی طراحی شده:

سیستم مدیریت هوشمند انرژی (SES)، یک شبیه ساز پیشرفته برای بهینه سازی مصرف انرژی در محیط های بسته (مانند اتاق های سرور یا محیط های اداری) است. عملکرد این سیستم در قالب یک چرخه پایش و تصمیم گیری خودکار تعریف می شود:

**پایش لحظه ای (Real-time Monitoring):** سیستم به صورت دوره ای و پیوسته، دمای محیط را از طریق سنسورها دریافت می کند. در این فرآیند، احتمال خرابی های سخت افزاری (سنسورها) در نظر گرفته شده و سیستم نسبت به آنها واکنش نشان می دهد.

**تحلیل و ارزیابی (Decision Logic):** داده های دریافت شده با تنظیمات ایده آل کاربر (Target Temperature) مقایسه می شوند. سیستم تصمیم می گیرد که آیا نیاز به مداخله (روشن/خاموش کردن سیستم سرمایشی) وجود دارد یا خیر.

**اجرای فرمان (Execution):** در صورت خروج دما از محدوده مجاز، سیستم دستورات لازم را به دستگاه های مربوطه ارسال کرده و بلافاصله وضعیت را تغییر می دهد.

ثبت و گزارش‌دهی (Logging & Reporting) : تمام تصمیمات، خطاها و تغییرات وضعیت، به همراه زمان وقوع، در فایل‌های لاگ ثبت می‌شوند. این داده‌ها در نهایت برای تحلیل آماری و تولید گزارش‌های عملکرد (Performance Reports) استفاده می‌شوند.

## معماری کلی سیستم (System Architecture)

این سیستم بر پایه معماری لایه لایه (Layered Architecture) طراحی شده است. هدف از این طراحی، کاهش وابستگی بین ماژول‌ها است تا هر بخش بتواند به صورت مستقل توسعه، تست و عیب‌یابی شود.

/alerts : مدیریت هوشمند هشدارهای سیستمی و اعلان‌های بحرانی (مانند خرابی سنسور).

/database : مدیریت لاگ‌ها و ثبت وقایع جهت مانیتورینگ و حسابرسی سیستم.

/devices : شامل کلاس‌های دستگاه‌های خروجی مانند (Cooling System) که دستورات را از موتور قوانین دریافت و اجرا می‌کنند.

/engine : قلب منطقی سیستم که شامل Rule Engine (تصمیم‌گیری)، Configuration (تنظیمات پویا) و Report (تحلیل عملکرد) است.

/sensors : رابط تعامل با محیط (شبیه‌سازی ورودی‌های سخت‌افزاری و دریافت داده‌ها).

/tests : شامل تست‌های واحد (Unit Tests) برای اطمینان از سلامت ماژول‌ها و پایداری کد.

/users : مدیریت کاربران، سطوح دسترسی و نقش‌های تعریف شده در سیستم.

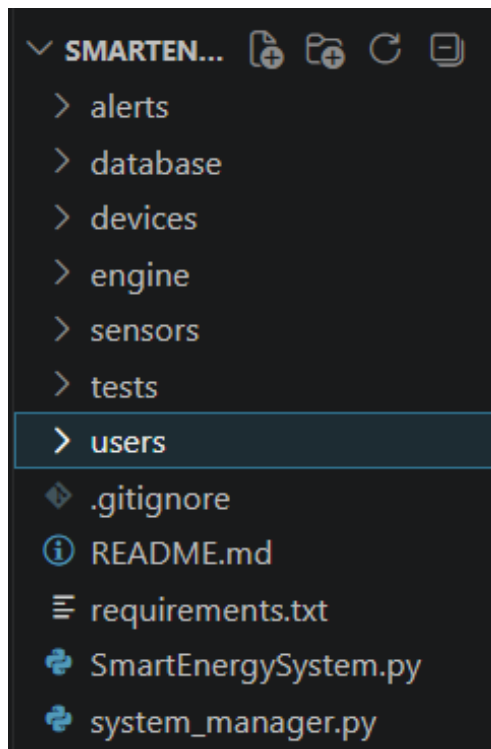
.gitignore : فایل پیکربندی گیت که از ورود فایل‌های غیرضروری (مانند \_\_pycache\_\_ یا فایل‌های موقت) به مخزن اصلی جلوگیری می‌کند تا تاریخچه کامیت‌ها تمیز بماند.

README.md : شناسنامه پروژه که راهنمای نصب و توضیحات فنی را برای کاربران دیگر فراهم می‌کند.

requirements.txt : لیستی از وابستگی‌های پروژه (در صورت نیاز به کتابخانه‌های جانبی).

SmartEnergySystem.py : هماهنگ‌کننده اصلی سیستم (Main Entry Point) که تمام کلاس‌ها را نمونه‌سازی کرده و چرخه اجرای برنامه را مدیریت می‌کند.

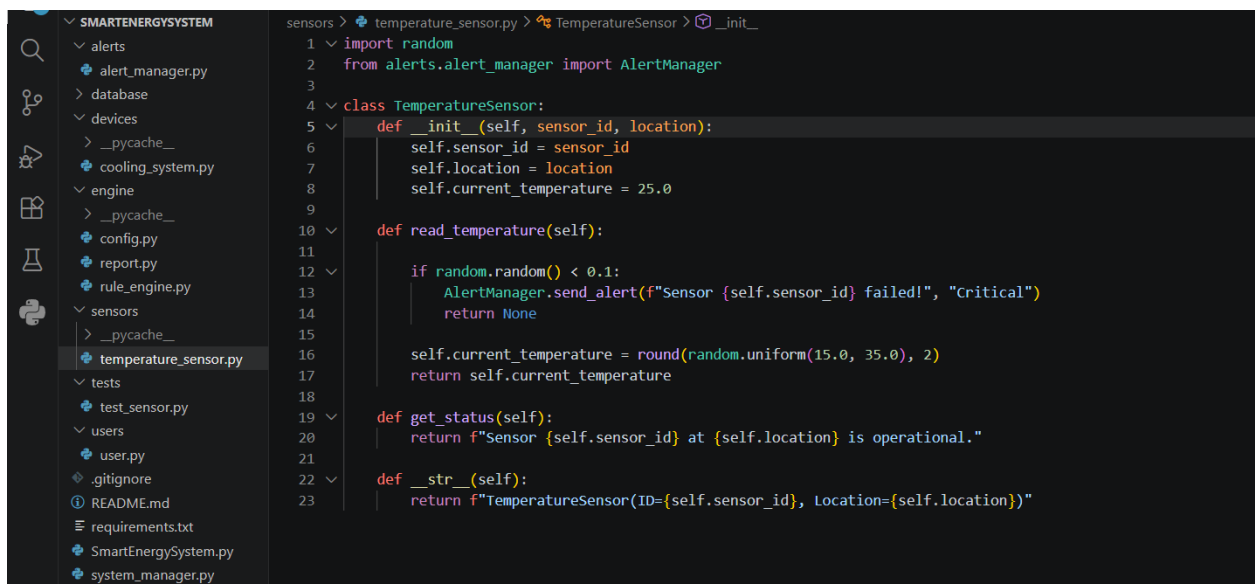
system\_manager.py : لایه کنترل‌کننده که وظیفه برقراری ارتباط بین موتور منطقی، لاگر و وضعیت فعلی سیستم را بر عهده دارد.



کد های منبع:

در ادامه به نمایش و تشریح کد های استفاده شده می رسیم و کامیت های ثبت شده مربوط به هر بخش را می آوریم.

لایه سنسور (sensors): نقطه شروع؛ جایی که داده های خام از محیط تولید می شوند.



این ماژل نقطه ی ورود داده به سیستم است و اهمیت بسیار زیادی در پایداری کل پروژه دارد.

## تحلیل فنی ماژول `sensors/temperature_sensor.py` :

این فایل مسئول شبیه سازی یک سنسور دمای واقعی است. در ادامه اجزای آن را باز می کنیم:

### ۱- تحلیل اجزای کلاس `TemperatureSensor`

- متد `--init--` (سازنده):

این متد هر نمونه از سنسور را با یک شناسه منحصر به فرد (`sensor_id`) و موقعیت مکانی (`location`) مقداردهی اولیه می کند. مقدار اولیه دما نیز روی ۲۵ درجه تنظیم شده است تا سیستم در حالت پیش فرض متعادل قرار بگیرد.

- متد `read_temperature` (قلب عملیاتی):

شبیه سازی خرابی (`Fault Simulation`): خط ۱۲ (`if random.random() < 0.1`) یکی از هوشمندانه ترین بخش های کد شماس است. این خط باعث می شود سنسور در ۱۰٪ مواقع به صورت تصادفی دچار خطا شود.

○ مدیریت بحران: به محض وقوع خطا، متد `AlertManager.send_alert` فراخوانی می شود تا سیستم از وضعیت بحرانی مطلع شود. این همان بحث تاب آوری یا `Robustness` است.

○ تولید داده: در حالت عادی، دما را با دقت ۲ رقم اعشار و در محدوده ۱۵ تا ۳۵ درجه تولید می کند (`random.uniform`).

- متد `get_status` و `--str--`:

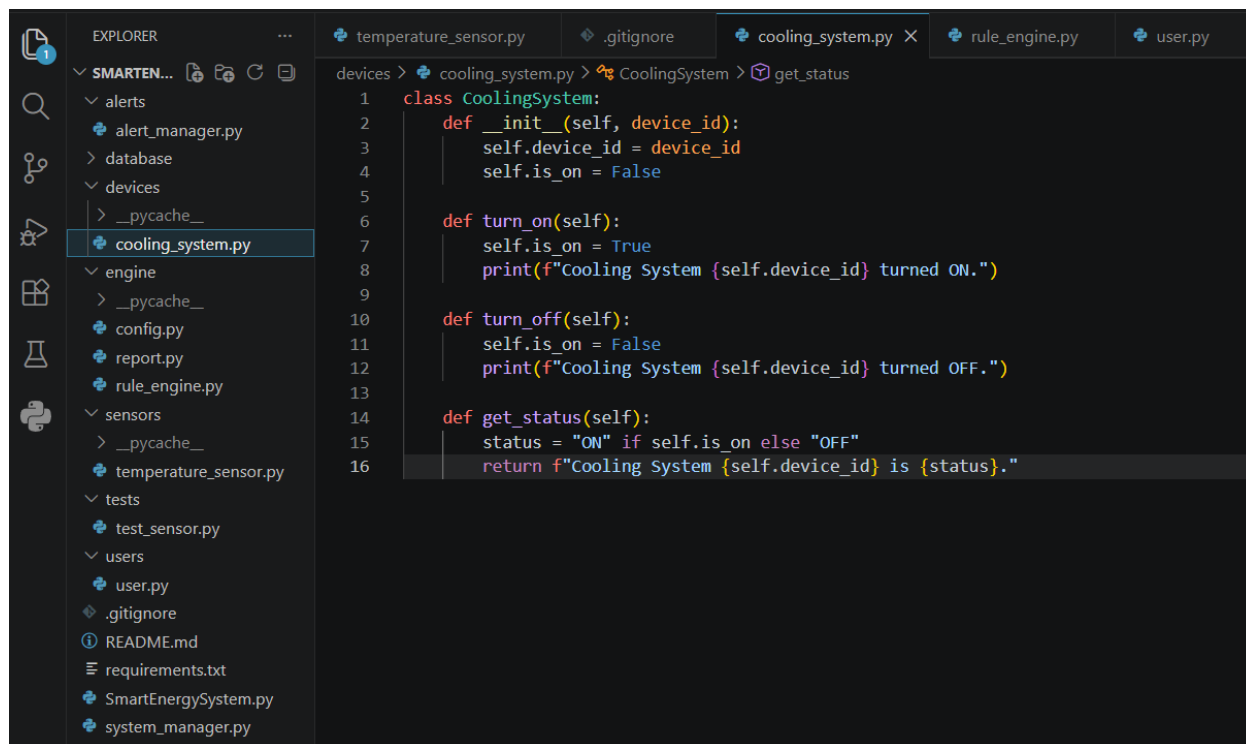
این متدها برای پایش وضعیت (`Monitoring`) هستند. متد `--str--` باعث می شود که اگر خواستید سنسور را در لاگ ها چاپ کنید، خروجی خوانایی مثل `Temperature Sensor(ID=1, Location=Livingroom)` دریافت کنید.

### ۲- نحوه کارکرد سیستم (`Logic Flow`)

وقتی سیستم اصلی می خواهد دما را بداند، فقط متد `read_temperature` را صدا می زند. سنسور به صورت خودکار خودش را بررسی می کند:

- آیا سالم؟ (چک کردن شانس ۱۰ درصدی خطا).
- اگر خرابیم: هشدار می فرستیم و `None` برمی گردانیم (تا موتور منطقی بداند نباید به این داده اعتماد کند).
- اگر سالمیم: دما را تولید و اعلام می کنم. این یعنی موتور منطقی سیستم (`RuleEngine`) هیچ وقت با داده های غلط (`Garbage Data`) کار نخواهد کرد!

لایه دستگاه ها (Devices): جایی که تغییرات فیزیکی اعمال می شود.



The screenshot shows a code editor with the following structure:

- EXPLORER
  - SMARTEN...
    - alerts
      - alert\_manager.py
    - database
    - devices
      - \_\_pycache\_\_
      - cooling\_system.py
    - engine
      - \_\_pycache\_\_
      - config.py
      - report.py
      - rule\_engine.py
    - sensors
      - \_\_pycache\_\_
      - temperature\_sensor.py
    - tests
      - test\_sensor.py
    - users
      - user.py
    - .gitignore
    - README.md
    - requirements.txt
    - SmartEnergySystem.py
    - system\_manager.py

The main editor shows the `cooling_system.py` file with the following code:

```
1 class CoolingSystem:
2     def __init__(self, device_id):
3         self.device_id = device_id
4         self.is_on = False
5
6     def turn_on(self):
7         self.is_on = True
8         print(f"Cooling System {self.device_id} turned ON.")
9
10    def turn_off(self):
11        self.is_on = False
12        print(f"Cooling System {self.device_id} turned OFF.")
13
14    def get_status(self):
15        status = "ON" if self.is_on else "OFF"
16        return f"Cooling System {self.device_id} is {status}."
```

این ماژول نقش «بازوهای اجرایی» سیستم را بازی می کند؛ یعنی همان جایی که تصمیمات اتخاذ شده توسط موتور منطقی، در دنیای واقعی (در اینجا به صورت شبیه سازی) نمود پیدا می کنند.

### تحلیل فنی ماژول `devices/cooling_system.py`:

این فایل کلاس `CoolingSystem` را در خود جای داده است که وظیفه ی کنترل وضعیت تجهیزات سرمایه یی را بر عهده دارد.

#### ۱- تحلیل اجزای کلاس `CoolingSystem`

- متد `--init--` (سازنده):

هر دستگاه با یک `device_id` منحصر به فرد شناخته می شود. ویژگی مهم آن، متغیر `self.is_on` است که به عنوان یک پرچم (Flag) وضعیت دستگاه (روشن یا خاموش) را در حافظه نگهداری می کند.

متدهای `turn_on` و `turn_off`: این متدها مستقیماً وضعیت دستگاه را تغییر می دهند. استفاده از `print` در اینجا برای «شبیه سازی خروجی سخت افزاری» است. در یک سیستم واقعی، این بخش می تواند دستور ارسال سیگنال به رله (Relay) برای روشن کردن کولر باشد.

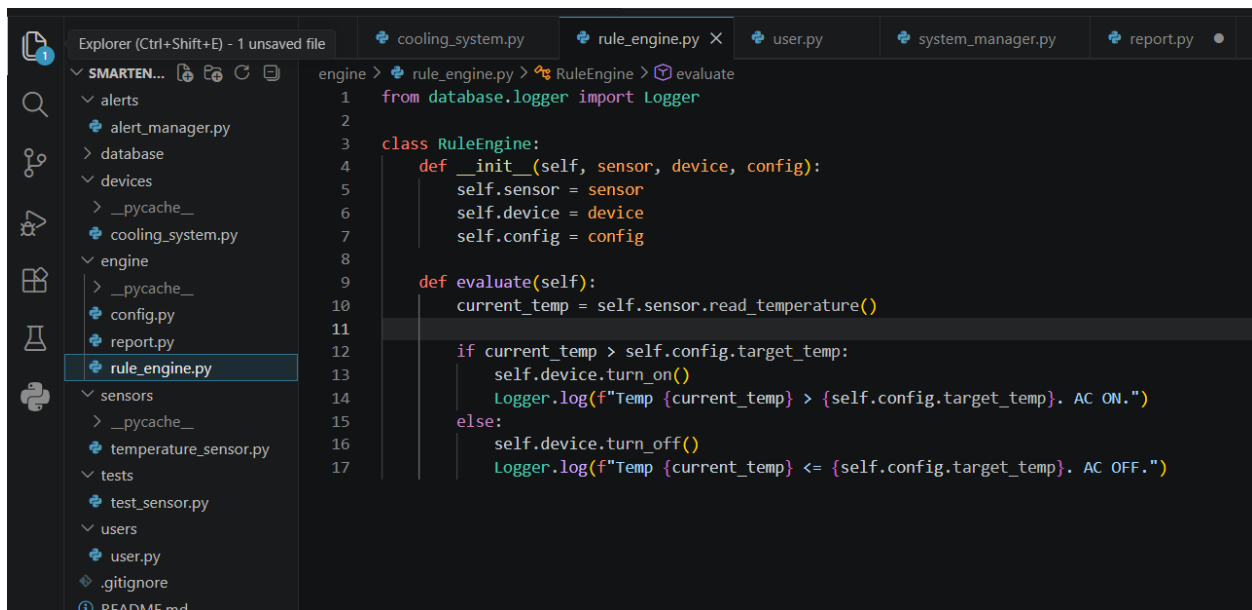
- **متد `get_status`:**

این متد وضعیت فعلی دستگاه را به صورت یک رشته‌ی خوانا برمی‌گرداند. این قابلیت برای گزارش‌گیری و مانیتورینگ لحظه‌ای سیستم بسیار کلیدی است.

## ۲- نحوه کارکرد سیستم (Logic Flow)

این ماژول از الگوی تغییر وضعیت (State Management) پیروی می‌کند.

- **دریافت فرمان:** موتور منطقی (RuleEngine) بر اساس دمای محیط، یکی از متدهای `turn_on` یا `turn_off` را صدا می‌زند.
- **به‌روزرسانی وضعیت:** دستگاه پرچم `self.is_on` را تغییر می‌دهد.
- **تأیید عملیات:** خروجی متناسب چاپ می‌شود تا لاگ سیستم (که در فایل `system_log.txt` ذخیره می‌شود) حاوی این تغییر وضعیت باشد.
- **لایه هوشمندی و منطق (Engine):** مغز سیستم، جایی که داده سنسور تحلیل شده و فرمان دستگاه صادر می‌شود.



```
1 from database.logger import Logger
2
3 class RuleEngine:
4     def __init__(self, sensor, device, config):
5         self.sensor = sensor
6         self.device = device
7         self.config = config
8
9     def evaluate(self):
10         current_temp = self.sensor.read_temperature()
11
12         if current_temp > self.config.target_temp:
13             self.device.turn_on()
14             Logger.log(f"Temp {current_temp} > {self.config.target_temp}. AC ON.")
15         else:
16             self.device.turn_off()
17             Logger.log(f"Temp {current_temp} <= {self.config.target_temp}. AC OFF.")
```

## تحلیل فنی ماژول `engine/rule_engine.py`:

این ماژول نقش میانجی تصمیم‌گیر را دارد. بدون این کلاس، سنسور فقط دما را می‌خواند و دستگاه فقط خاموش/روشن می‌شد؛ اما اینجا، این دو بدون اینکه همدیگر را بشناسند، با هم همکاری می‌کنند.

## ۱- تحلیل اجزای کلاس Rule Engine

- متد `--init--` (سازنده و پیونددهنده):

این متد با دریافت نمونه‌هایی از سنسور، دستگاه و تنظیمات (`config`)، آن‌ها را در خودش ذخیره می‌کند. این یعنی موتور، وابستگی‌های خود را از بیرون دریافت می‌کند که به آن `Dependency Injection` می‌گویند.

- متد `evaluate` (هسته پردازشی):

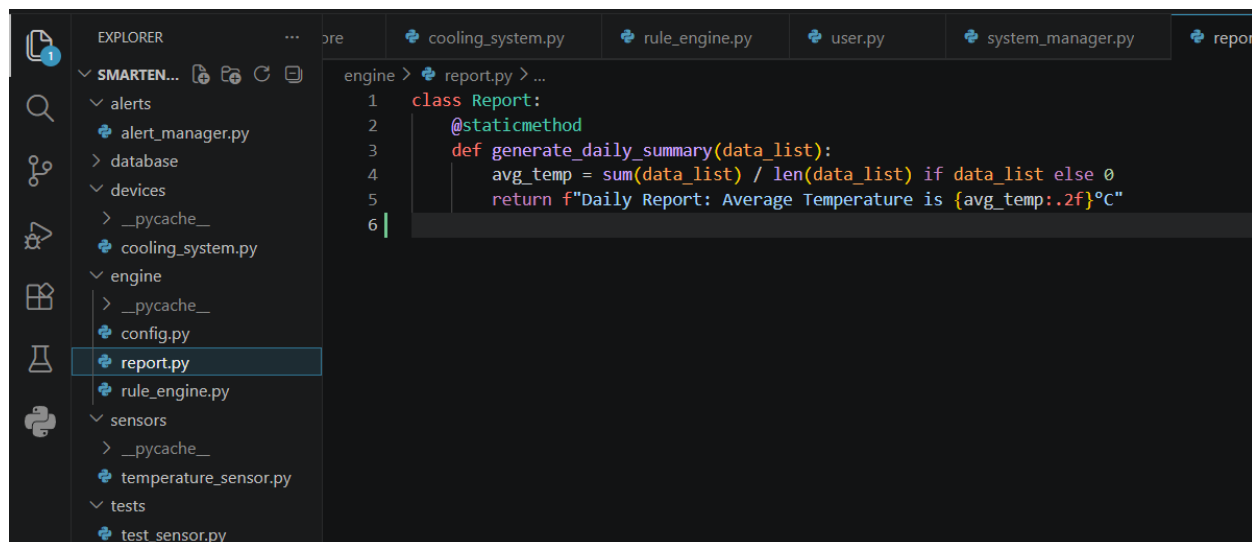
- دریافت داده (Line 10): دما را از سنسور می‌گیرد.
- منطق تصمیم‌گیری (Lines 12-17): اگر دمای فعلی از `target_temp` (که در `config` تعریف شده) بیشتر باشد، فرمان `turn_on` را به دستگاه صادر می‌کند.
- ثبت وقایع (Logging): در هر مرحله، تصمیم گرفته شده را به کلاس `Logger` می‌فرستد تا در تاریخچه سیستم ثبت شود. این برای ممیزی (`Audit`) بسیار مهم است.

## ۲- تحلیل مهندسی طراحی

- کاهش وابستگی (`Decoupling`): با توجه به این نکته که `RuleEngine` اصلاً نمی‌داند سنسور چگونه دما را

می‌خواند (API یا `2C` است) و نمی‌داند دستگاه چگونه روشن می‌شود (رله است یا سیگنال). او فقط می‌گوید: «دما بالا است، دستگاه، روشن شو!». این ویژگی باعث می‌شود سیستم شما به راحتی در آینده قابل ارتقا باشد.

- پیکربندی پویا (`Dynamic Config`): به جای اینکه عدد ۲۵ (دمای هدف) را داخل کد بنویسیم، آن را از `self.config.target_temp` می‌خوانیم. این یعنی اگر بخواهیم سیستم را برای یک سردخانه با دمای ۵ درجه تنظیم کنیم، کافیه `config` را عوض کنیم، نه کل کد موتور را.



```
1 class Report:
2     @staticmethod
3     def generate_daily_summary(data_list):
4         avg_temp = sum(data_list) / len(data_list) if data_list else 0
5         return f"Daily Report: Average Temperature is {avg_temp:.2f}°C"
6
```

## تحلیل فنی ماژول engine/report.py :

این ماژول اگرچه ساده به نظر می‌رسد، اما از نظر مهندسی بسیار مهم است؛ زیرا وظیفه دارد داده‌های خام جمع‌آوری شده را به «اطلاعات قابل فهم» تبدیل کند.

### ۱- تحلیل اجزای کلاس Report

- استفاده از `@staticmethod`: از آنجا که این کلاس نیازی به ذخیره وضعیت داخلی (State) ندارد (یعنی گزارش‌گیر نیاز نیست بداند دمای فعلی چیست، فقط لیستی از داده‌ها را می‌گیرد و پردازش می‌کند)، استفاده از متد استاتیک باعث می‌شود که این متد بدون نیاز به ساخت نمونه (Instance) از کلاس، در هر جای برنامه (مثلاً در `system_manager`) قابل فراخوانی باشد.

- متد `generate_daily_summary`:

- پردازش منطقی (Line 4): استفاده از عبارات شرطی فشرده (Ternary Operator) باعث شده تا از خطای احتمالی `ZeroDivisionError` (تقسیم بر صفر در صورتی که لیستی از دماها وجود نداشته باشد) جلوگیری شود.

- فرمت‌دهی خروجی: با استفاده از `f-string` و فرمت `{avg_temp:.2f}`، خروجی گزارش با دقت دو رقم اعشار استانداردسازی شده است.

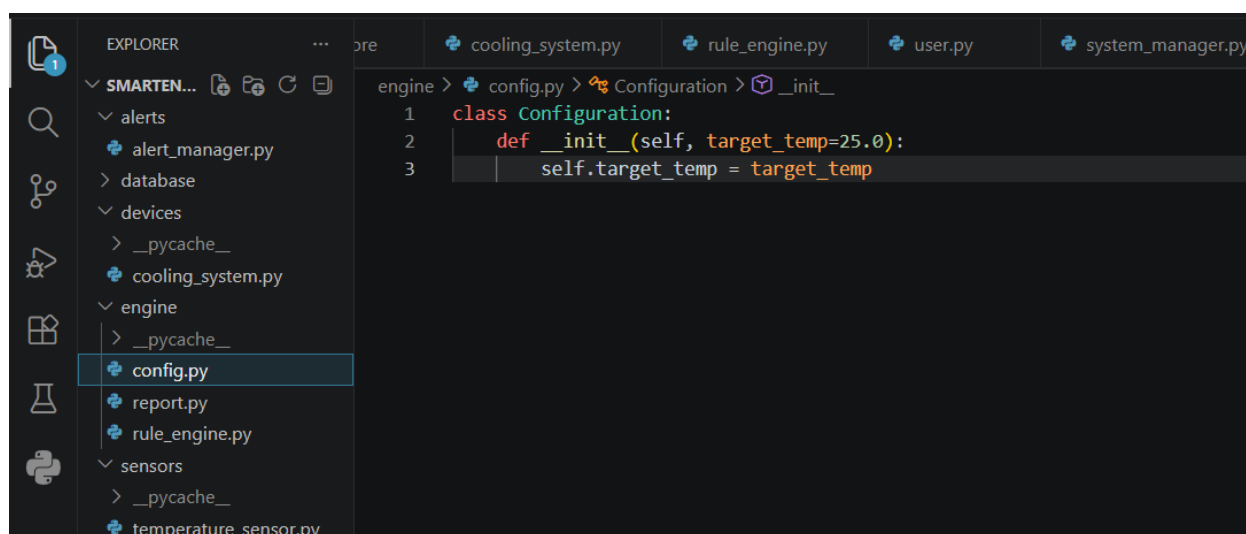
### ۲- تحلیل مهندسی طراحی

- تفکیک ارائه از منطق (Presentation vs. Logic): موتور قوانین (Rule Engine) فقط مسئول «تصمیم‌گیری آنی» است، اما این کلاس مسئول «تحلیل بلندمدت» است. این جداسازی به شما اجازه می‌دهد که در آینده،



بدون دست زدن به کدهای موتور قوانین، این کلاس را پیچیده‌تر کنید (مثلاً رسم نمودار یا ارسال گزارش به ایمیل).

- قابلیت تست‌پذیری: چون این متد یک ورودی می‌گیرد و یک خروجی برمی‌گرداند، نوشتن تست برای آن (Unit Testing) بسیار ساده است.



### تحلیل فنی ماژول engine/config.py :

این فایل کلاس Configuration را در خود جای داده است. وظیفه اصلی آن نگهداری «پارامترهای عملیاتی» سیستم است.

#### ۱- تحلیل اجزای کلاس Configuration

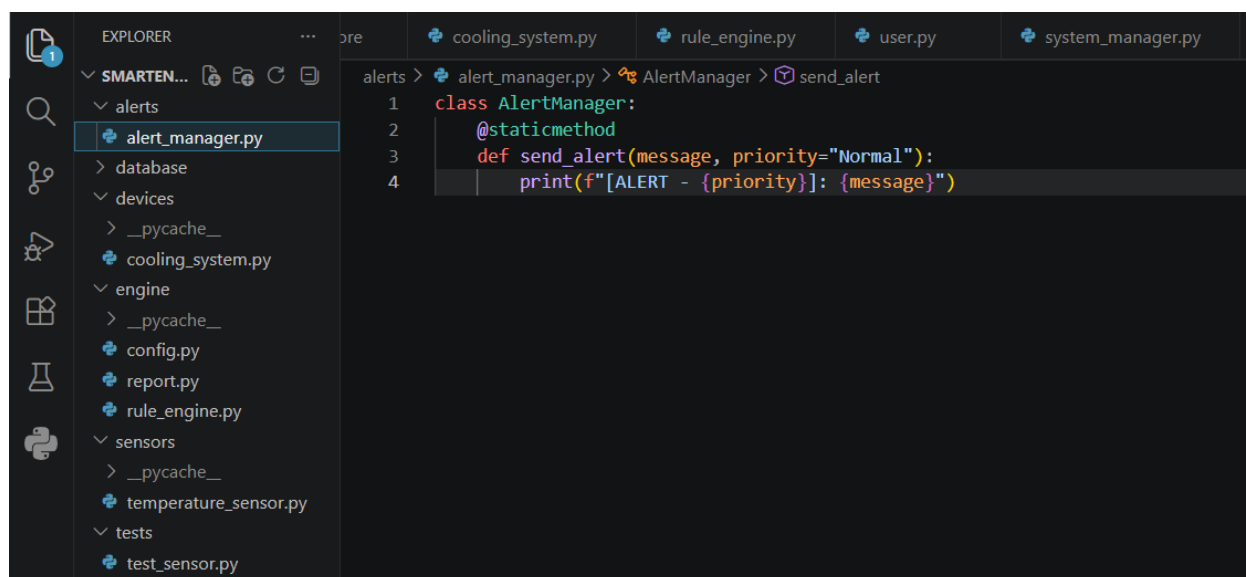
- متد `--init--` (سازنده با مقدار پیش‌فرض):
  - این متد مقدار `target_temp` را با مقدار پیش‌فرض 25.0 می‌گیرد.
  - مقداردهی اولیه (Initialization): با ذخیره کردن این مقدار در `self.target_temp`، این متغیر به عنوان «نقطه مرجع» برای کل سیستم شناخته می‌شود.

#### ۲- تحلیل مهندسی طراحی

- **جلوگیری از Hard-coding**: اگر به جای این کلاس، عدد ۲۵.۰ را مستقیماً داخل `RuleEngine` می‌نوشتید، برای تغییر دمای هدف، مجبور می‌شدید کد اصلی را تغییر دهید و دوباره کامپایل کنید. حالا شما می‌توانید با تغییر یک پارامتر در زمان اجرا (Runtime)، رفتار کل سیستم را عوض کنید.

- **تسهیل در تست (Testing):** در هنگام نوشتن تست‌های واحد (test\_sensor.py)، می‌توانید یک شیء از Configuration با دمای دلخواه بسازید و آن را به موتور قوانین بدهید تا سناریوهای مختلف (مثل دمای خیلی سرد یا خیلی گرم) را تست کنید.
- **پایه توسعه‌پذیری:** این کلاس اکنون فقط دمای هدف را نگه می‌دارد، اما در آینده می‌تواند پارامترهای دیگری مثل «بازه زمانی پایش»، «آستانه تحمل خطا» یا «سطح دسترسی کاربران» را نیز در خود جای دهد بدون اینکه کل معماری سیستم به هم بریزد.

**لایه پشتیبان (Supportive Layers):** سرویس‌های خدماتی که در تمام مراحل حضور دارند.



### تحلیل فنی ماژول alerts/alert\_manager.py :

این فایل کلاس AlertManager را در خود جای داده است. وظیفه اصلی آن، اطلاع‌رسانی در لحظه، هنگام بروز وقایع مهم (مانند خطای سنسور) است.

#### ۱- تحلیل اجزای کلاس AlertManager

- استفاده از @staticmethod: مشابه ماژول گزارش‌گیری، در اینجا هم از متد استاتیک استفاده شده است. از آنجایی که «ارسال هشدار» یک عمل مستقل است و نیاز به حفظ وضعیت (State) در داخل کلاس ندارد، استفاده

از استاتیک باعث می‌شود هر کجای پروژه (در موتور قوانین، سنسورها یا مدیر سیستم) بتوانیم بدون نیاز به ساخت نمونه، هشدار صادر کنیم.

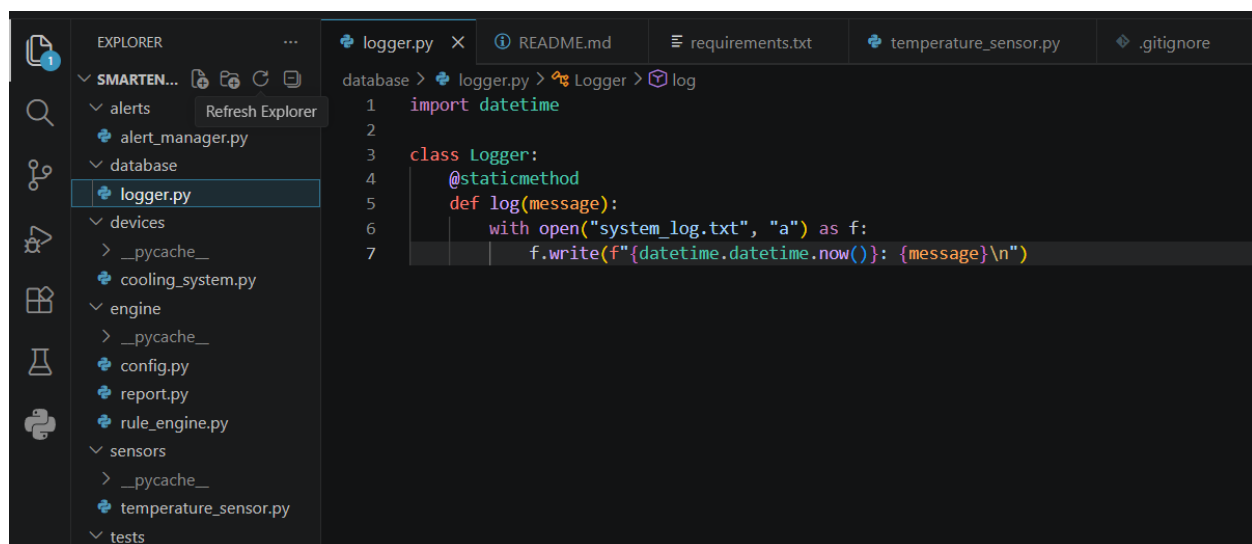
- **متد: send\_alert**

- **پارامتر message:** متن اصلی هشدار را دریافت می‌کند.
- **پارامتر priority:** با تعیین سطح اهمیت (Normal, Critical, Warning) به سیستم اجازه می‌دهد که در آینده، برای هشدارهای بحرانی (مثل آتش‌سوزی یا خرابی سنسور) واکنش‌های متفاوتی نشان دهد.
- **خروجی:** در حال حاضر با دستور `print` هشدار را در کنسول نمایش می‌دهد. در نسخه‌های توسعه‌یافته، این بخش می‌تواند هشدار را به ایمیل، پیامک یا پنل مدیریتی بفرستد.

## ۲- تحلیل مهندسی طراحی

- **توزیع آگاهی (Event Notification):** این کلاس یک نقطه مرکزی (Centralized Point) برای تمامی هشدارهای سیستم است. اگر بخواهید در آینده سیستم را ارتقا دهید (مثلاً اضافه کردن قابلیت ارسال ایمیل هنگام خطای سنسور)، فقط کافی است کد داخل همین متد را تغییر دهید و نیازی نیست به صدها جای دیگر پروژه دست بزنید.

- **استانداردسازی (Standardization):** تمام هشدارهای سیستم (از سنسور تا موتور قوانین) با یک فرمت یکسان `message [ALERT - priority]`: نمایش داده می‌شوند که کار خواندن لاگ‌ها را برای مدیر سیستم بسیار راحت می‌کند.



```
1 import datetime
2
3 class Logger:
4     @staticmethod
5     def log(message):
6         with open("system_log.txt", "a") as f:
7             f.write(f"{datetime.datetime.now()}: {message}\n")
```

## تحلیل فنی ماژول database/logger.py :

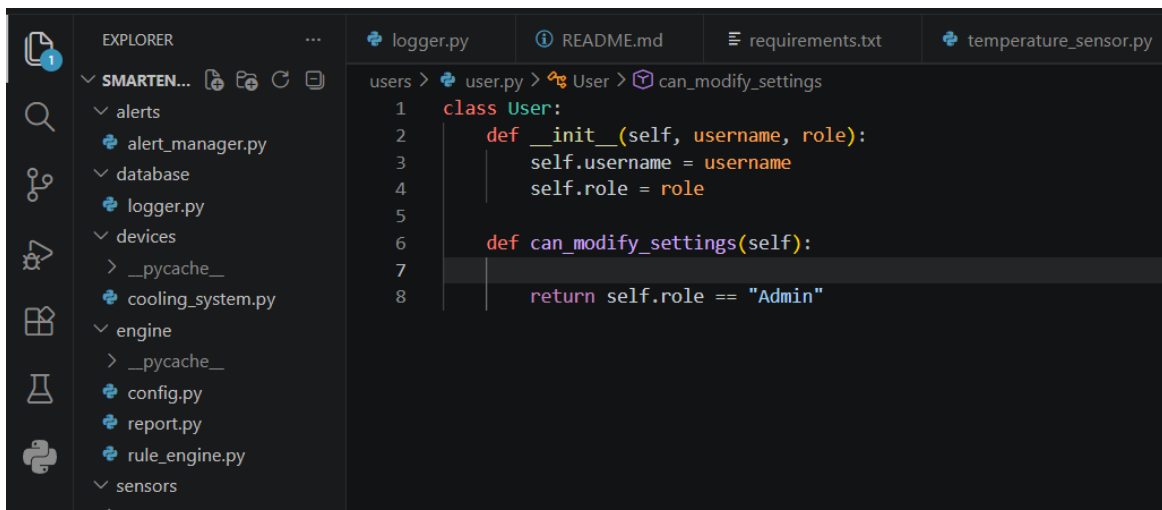
این ماژول از کلاس Logger برای ثبت تمامی اتفاقات در یک فایل متنی استفاده می‌کند.

### ۱- تحلیل اجزای کلاس Logger

- استفاده از `@staticmethod`: مشابه ماژول‌های قبلی، اینجا هم از متد استاتیک استفاده شده است. چون ثبت یک واقعه عملیاتی است که به وضعیت داخلی یک شیء (Object) وابسته نیست، این طراحی اجازه می‌دهد که در هر نقطه از برنامه (مثلاً در RuleEngine یا AlertManager) بدون نیاز به ساخت نمونه، فقط با نوشتن `Logger.log("message")` واقعه را ثبت کنید.
- استفاده از `with open(...)` (مدیریت منابع): با باز کردن فایل در مد "a" (یعنی Append یا اضافه کردن)، مطمئن می‌شویم که لاگ‌های قبلی پاک نمی‌شوند و هر واقعه جدید به انتهای فایل `system_log.txt` اضافه می‌شود.
- ثبت زمان دقیق (`datetime.datetime.now`): هر لاگ با مهر زمانی (Timestamp) ثبت می‌شود. این یعنی مدیر سیستم می‌تواند در آینده دقیقاً متوجه شود که در ساعت ۳ بامداد، سیستم چه تصمیمی گرفته یا چه خطایی رخ داده است.

### ۲- تحلیل مهندسی طراحی

- پایداری اطلاعات (Data Persistence): اگر سیستم به هر دلیلی کرش کند یا خاموش شود، لاگ‌هایی که تا آن لحظه ثبت شده‌اند در فایل `system_log.txt` باقی می‌مانند. این برای عیب‌یابی (Debugging) فوق‌العاده است.
- تسهیل در ممیزی (Auditability): تمام تصمیمات RuleEngine و تمام هشدارهای AlertManager در اینجا متمرکز می‌شوند. این یعنی شما یک «جعبه سیاه» (Black Box) برای سیستم هوشمند خود ساخته‌اید.



## تحلیل فنی ماژول `users/user.py` :

این ماژول وظیفه مدل‌سازی موجودیت کاربر و مدیریت سطوح دسترسی در سیستم را بر عهده دارد. این بخش، پایه و اساس امنیت سیستم برای جلوگیری از دسترسی‌های غیرمجاز به تنظیمات حساس است.

### ۱- تحلیل اجزای کلاس `User`

- متد `--init--` (سازنده):

این متد هر نمونه از کاربر را با یک نام کاربری (`username`) و نقش کاربری (`role`) مقداردهی اولیه می‌نماید. انتساب نقش به هر کاربر، امکان پیاده‌سازی سیستم‌های کنترل دسترسی مبتنی بر نقش (`RBAC`) را فراهم می‌آورد.

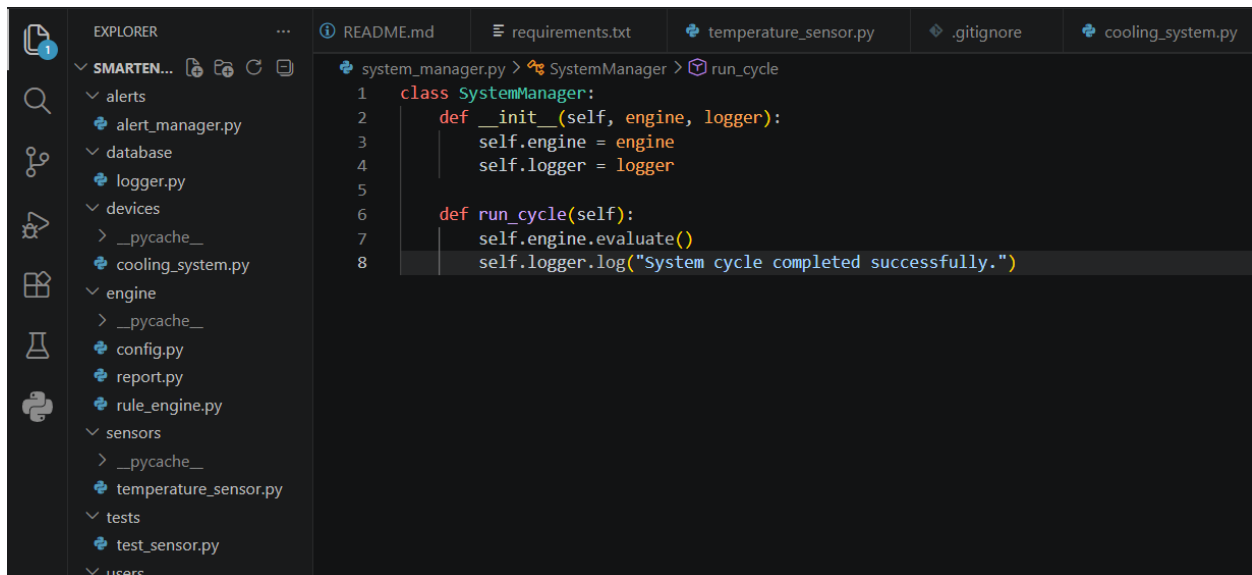
- متد `can_modify_settings`:

این متد یک منطق شرطی ساده برای اعتبارسنجی مجوزها ارائه می‌دهد. بر اساس این متد، تنها کاربرانی با نقش `Admin` اجازه اعمال تغییرات در پیکربندی‌های سیستم را خواهند داشت. این رویکرد، لایه‌ای از امنیت و یکپارچگی را برای تنظیمات حساس ساختمان ایجاد می‌کند.

### ۲- اهمیت مهندسی این ماژول

- اصل حداقل امتیاز (**Principle of Least Privilege**) : با تفکیک نقش‌ها، سیستم اطمینان حاصل می‌کند که کاربران تنها به قابلیت‌هایی دسترسی دارند که برای آن نقش تعریف شده است. این امر احتمال بروز خطاهای انسانی یا تغییرات ناخواسته در عملکرد بهینه‌سازی انرژی را به حداقل می‌رساند.
- قابلیت توسعه : این ساختار ساده، بستر لازم برای پیاده‌سازی سیستم‌های احراز هویت پیچیده‌تر در نسخه‌های آتی را فراهم می‌آورد، بدون اینکه نیاز به تغییر ساختار اصلی کلاس باشد.

## لایه هماهنگ کننده: (System Integration)



### تحلیل فنی ماژول system\_manager.py :

این ماژول لایه‌ی کنترل کننده‌ی سطح بالاست که وظیفه‌ی یکپارچه‌سازی و مدیریت چرخه‌ی حیات عملیاتی سیستم را بر عهده دارد.

#### ۱- تحلیل اجزای کلاس SystemManager

- متد `--init--` (سازنده):

این کلاس به جای درگیر شدن با جزئیات پیاده‌سازی موتور یا لاگر، آن‌ها را از طریق مکانیزم تزریق وابستگی (Dependency Injection) دریافت می‌کند. این طراحی تضمین می‌کند که `SystemManager` کاملاً نسبت به پیاده‌سازی داخلی ماژول‌های زیرمجموعه ناآگاه (Loose Coupling) باقی بماند.

- متد `run_cycle`: این متد «نقطه کانونی اجرای عملیات» است.

- **فراخوانی منطق:** ابتدا `self.engine.evaluate()` را فراخوانی می‌کند تا فرآیند تصمیم‌گیری (پایش دما و کنترل دستگاه) انجام شود.
- **تأیید چرخه:** پس از اتمام موفقیت‌آمیز تصمیم‌گیری، با فراخوانی `self.logger.log(...)` پایان چرخه‌ی عملیاتی را ثبت می‌کند.

## ۲- اهمیت مهندسی این ماژول

- **انتزاع (Abstraction) :** کلاس `SystemManager` به عنوان یک نمای بیرونی (Facade) برای کل پروژه عمل می‌کند. یعنی اگر کسی بخواهد سیستم را اجرا کند، کافایت فقط با این کلاس در تعامل باشد و نیازی نیست درگیر پیچیدگی کلاس‌های سنسور یا موتور قوانین شود.
- **تضمین نظم اجرا:** این ماژول تضمین می‌کند که تمامی مراحل یک چرخه‌ی پایش (پایش -> تصمیم‌گیری -> لاگ‌گیری) با ترتیب صحیح و منطقی اجرا شوند.

```
SmartEnergySystem.py > ...
1 # Smart Energy System - Version 1.0.0
2 # Developed for Software Engineering Project
3 from sensors.temperature_sensor import TemperatureSensor
4 from devices.cooling_system import CoolingSystem
5 from engine.rule_engine import RuleEngine
6
7 my_sensor = TemperatureSensor(sensor_id="S01", location="Server Room")
8 my_cooling = CoolingSystem(device_id="AC01")
9
10 engine = RuleEngine(sensor=my_sensor, device=my_cooling)
11
12 print("--- Smart Energy System Started ---")
13 engine.evaluate()
14 print("--- Cycle Finished ---")
```

## تحلیل فنی ماژول `SmartEnergySystem.py` :

این فایل به عنوان لایه‌ی نهایی انتزاع، وظیفه‌ی مقداردهی اولیه (Initialization) و اجرای چرخه‌ی عملیاتی پروژه را بر عهده دارد. این فایل نمایی بیرونی از سیستم است که پیچیدگی‌های داخلی ماژول‌ها را در خود پنهان کرده است.

### ۱- تحلیل عملکرد ماژول

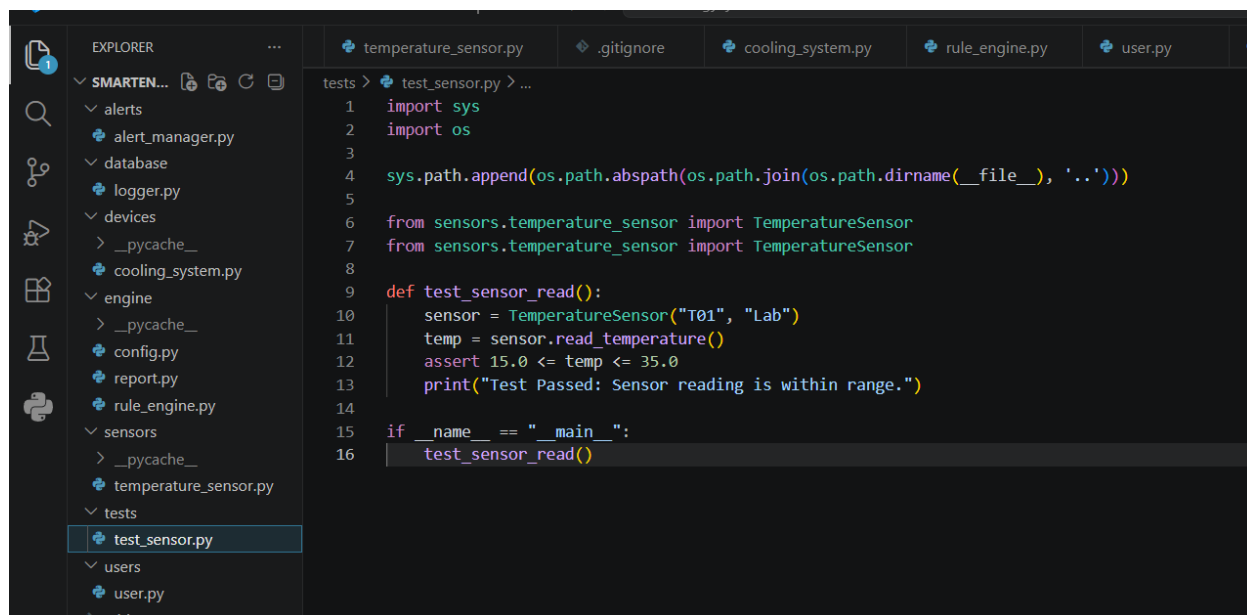
- **واردسازی وابستگی‌ها (Imports) :** این بخش نشان‌دهنده‌ی یکپارچگی ماژول‌های پروژه است. با وارد کردن کلاس‌های `TemperatureSensor`، `CoolingSystem` و `RuleEngine` از دایرکتوری‌های مربوطه، تمامی اجزای پراکنده در یک فضای نام واحد گرد هم آمده‌اند.
- **تزریق وابستگی (Dependency Injection) :** در خط ۱۰، شما با نمونه‌سازی از `RuleEngine` و ارسال `my_sensor` و `my_cooling` به آن، الگوی طراحی تزریق وابستگی را به شکلی کاملاً گویا پیاده‌سازی کرده‌اید. این باعث می‌شود که `RuleEngine` مستقلاً از نحوه تولید سنسور یا دستگاه آگاه نباشد.

- اجرای چرخه‌ی عملیاتی (Execution) : فراخوانی متد engine.evaluate() در خط ۱۳، لحظه‌ی طلایی سیستم است؛ جایی که تمام ماژول‌های طراحی شده با هم وارد تعامل شده و هوشمندی سیستم انرژی ساختمان را به فعلیت می‌رسانند.

## ۲- اهمیت مهندسی این ماژول

- سادگی برای کاربر (Usability) : این طراحی باعث شده است که اجرای سیستم برای کاربر نهایی یا ادمین ساختمان بسیار ساده باشد. تنها با یک دستور ساده (python SmartEnergySystem.py) ، کل چرخه‌ی پیچیده پایش و مدیریت انرژی آغاز می‌شود.
- قابلیت گسترش (Maintainability) : اگر در آینده بخواهید سنسور دومی اضافه کنید یا موتور منطقی جدیدی جایگزین کنید، تغییرات تنها در این فایل هماهنگ‌کننده اعمال می‌شود و نیازی به بازنویسی منطق داخلی سایر کلاس‌ها نیست.

لایه تضمین کیفیت (QA Layer): این لایه از صحت عملکرد بخش‌های حیاتی سیستم اطمینان حاصل می‌کند



```

EXPLORER
├── SMARTEN...
│   ├── alerts
│   │   └── alert_manager.py
│   ├── database
│   │   └── logger.py
│   ├── devices
│   │   ├── __pycache__
│   │   └── cooling_system.py
│   ├── engine
│   │   ├── __pycache__
│   │   ├── config.py
│   │   ├── report.py
│   │   ├── rule_engine.py
│   ├── sensors
│   │   ├── __pycache__
│   │   ├── temperature_sensor.py
│   │   └── tests
│   │       └── test_sensor.py
│   ├── users
│   │   └── user.py
│   └── .gitignore
├── temperature_sensor.py
├── .gitignore
├── cooling_system.py
├── rule_engine.py
└── user.py

tests > test_sensor.py > ...
1  import sys
2  import os
3
4  sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
5
6  from sensors.temperature_sensor import TemperatureSensor
7  from sensors.temperature_sensor import TemperatureSensor
8
9  def test_sensor_read():
10     sensor = TemperatureSensor("T01", "Lab")
11     temp = sensor.read_temperature()
12     assert 15.0 <= temp <= 35.0
13     print("Test Passed: Sensor reading is within range.")
14
15 if __name__ == "__main__":
16     test_sensor_read()
  
```

## تحلیل فنی ماژول tests/test\_sensor.py :

این ماژول مسئول تست واحد (Unit Test) برای ماژول سنسور است. هدف از این تست، اعتبارسنجی خروجی‌های سنسور در شرایط عملیاتی است.



## ۱- تحلیل اجزای ماژول تست

- **تنظیم مسیر (Line 4):** با استفاده از `sys.path.append (...)` به پایتون می‌گوییم که چگونه دایرکتوری‌های پروژه را پیدا کند تا بتواند ماژول‌های سنسور را از پوشه‌ی `sensors` به راحتی وارد (`Import`) کند. این یک تکنیک استاندارد برای اجرای تست‌های مجزا در پروژه‌های ساختاریافته است.
- **متد تست `test_sensor_read`:**
  - **شبیه‌سازی واقعی:** یک نمونه سنسور با مشخصات معین ("`T01`", "`Lab`") ساخته می‌شود.
  - **تأیید خروجی (Assertion):** با استفاده از دستور `assert 15.0 <= temp <= 35.0` بررسی می‌شود که آیا دمای تولیدشده توسط سنسور در محدوده‌ی مجاز تعریف شده قرار دارد یا خیر. اگر دما خارج از این محدوده باشد، سیستم تست بلافاصله متوقف شده و خطا می‌دهد.

## ۲- تحلیل مهندسی طراحی

- **تضمین کیفیت (Quality Assurance):** وجود تست، یعنی سیستم شما بر پایه‌ی حدس و گمان نیست؛ بلکه با داده‌های واقعی صحت‌سنجی شده است.
- **پیشگیری از رگرسیون (Regression Testing):** اگر در آینده کدهای مربوط به سنسور را تغییر دهید، تنها با اجرای این فایل تست، می‌توانید مطمئن شوید که تغییرات جدید، منطق اصلی سنسور را خراب نکرده است.

## فایل README



```
1  # Smart Energy System
2  . این پروژه یک سیستم مدیریت هوشمند انرژی ساختمان است که بر اساس معماری ماژولار طراحی شده است.
3
4  ## نحوه اجرا:
5  python SmartEnergySystem.py
6
7  ## ویژگی‌ها:
8  - پایش لحظه‌ای دما
9  - کنترل هوشمند دستگاه‌های سرمایشی
10 - ثبت وقایع (Logging)
11 - سیستم هشدار (Alerts)
```

این فایل شامل کلیتی از سیستم است و مواردی مثل ویژگی‌ها و نحوه‌ی اجرای آن آورده شده است.

## تاریخچه کامیت ها

این پروژه شامل ۲۰ کامیت اصلی و دارای معنا می باشد .

```
PS G:\777\me\SmartEnergySystem> git log --oneline --graph --all
* 90472f3 (HEAD -> master) Final project delivery: Version 1.0.0 stable
* e87f566 Add .gitignore to clean up repository
* 48ac715 Add __str__ method for better object representation
* 10a88b9 Add project documentation in README
* 835b1b8 Add Report class for performance analysis
* 4374f76 Add Report class for performance analysis
* b7ff7a5 Add SystemManager to coordinate system components
* 5f53a3b Update User class with permission logic
* 3ab0ca5 Add error detection and alert trigger in sensor
* 1ad89ba Add AlertManager class for system notifications
* 1fefb73 Fix module import in tests and verify sensor
* 91aac55 Add unit test for TemperatureSensor
* fbdd2f7 Update RuleEngine to use dynamic configuration
* d45b4e3 Add User class for role management
* 0573817 Add Configuration class for target temperature
* ec095a9 Add Configuration class for target temperature
* 1e847a1 Add User class for role management
* de7a518 Integrate Logger with RuleEngine
* 9a56a75 Add Logger class for event tracking
* dbf647d Complete prototype integration
* 1dcbb0a Add TemperatureSensor class
* 9b83155 Add RuleEngine class
* 9dc2131 Add CoolingSystem class
* d0601a0 Add RuleEngine class
* e787f34 Add CoolingSystem class
* 1b4dddb Add TemperatureSensor class
* 958c146 Initialize project structure
(END)
```

|                                             |   |
|---------------------------------------------|---|
| • Add User class for role management Hasti  |   |
| • user.py users                             | A |
| • Integrate Logger with RuleEngine Hasti    |   |
| • rule_engine.py engine                     | M |
| • Add Logger class for event tracking Hasti |   |
| • logger.py database                        | A |
| • Complete prototype integration Hasti      |   |
| • SmartEnergySystem.py                      | M |
| • Add TemperatureSensor class Hasti         |   |
| • temperature_sensor.py sensors             | M |
| • Add RuleEngine class Hasti                |   |
| • rule_engine.py engine                     | M |
| • Add CoolingSystem class Hasti             |   |
| • cooling_system.py devices                 | M |
| • Add RuleEngine class Hasti                |   |
| • rule_engine.py engine                     | A |
| • Add CoolingSystem class Hasti             |   |
| • cooling_system.py devices                 | A |
| • Add TemperatureSensor class Hasti         |   |
| • temperature_sensor.py sensors             | A |
| • Initialize project structure Hasti        |   |
| • README.md                                 | A |
| • SmartEnergySystem.py                      | A |
| • requirements.txt                          | A |

|                                                           |   |
|-----------------------------------------------------------|---|
| • Add Report class for performance analysis Hasti         |   |
| • report.py engine                                        | M |
| • Add Report class for performance analysis Hasti         |   |
| • report.py engine                                        | A |
| • Add SystemManager to coordinate system components Hasti |   |
| • system_manager.py                                       | A |
| • Update User class with permission logic Hasti           |   |
| • user.py users                                           | M |
| • Add error detection and alert trigger in sensor Hasti   |   |
| • temperature_sensor.py sensors                           | M |
| • Add AlertManager class for system notifications Hasti   |   |
| • alert_manager.py alerts                                 | A |
| • Fix module import in tests and verify sensor Hasti      |   |
| • test_sensor.py tests                                    | M |
| • Add unit test for TemperatureSensor Hasti               |   |
| • test_sensor.py tests                                    | A |
| • Update RuleEngine to use dynamic configuration Hasti    |   |
| • rule_engine.py engine                                   | M |
| • Add User class for role management Hasti                |   |
| • user.py users                                           | M |
| • Add Configuration class for target temperature Hasti    |   |
| • config.py engine                                        | M |
| • Add Configuration class for target temperature Hasti    |   |
| • config.py engine                                        | A |

|                                                             |          |
|-------------------------------------------------------------|----------|
| ○ Final project delivery: Version 1.0.0 stable Hasti        | 🎯 master |
| • SmartEnergySystem.py                                      | M        |
| • Add .gitignore to clean up repository Hasti               |          |
| • .gitignore                                                | A        |
| • Add __str__ method for better object representation Hasti |          |
| • temperature_sensor.py sensors                             | M        |
| • Add project documentation in README Hasti                 |          |
| • README.md                                                 | M        |

## مستندات ساخت و استقرار (Build & Deployment Documentation)

این سیستم با هدف حداکثر سادگی در اجرا و قابلیت حمل بالا (Portability) طراحی شده است. به دلیل استفاده از کتابخانه‌های استاندارد پایتون، پروژه نیازی به نصب وابستگی‌های پیچیده (Dependencies) ندارد.

### ۱- پیش‌نیازها (Prerequisites)

برای اجرای این پروژه، تنها نیاز به موارد زیر است:

- **مفسر پایتون:** نسخه ۳.۸ یا بالاتر. (Python 3.8+)
- **محیط اجرا:** ترمینال یا خط فرمان (CMD/Terminal) در ویندوز، لینوکس یا مک.

### ۲- نحوه راه‌اندازی و اجرا (Quick Start)

پس از دریافت فایل‌های پروژه، مراحل زیر را برای استقرار دنبال کنید:

- **بررسی ساختار:** اطمینان حاصل کنید که تمام پوشه‌ها (sensors/, devices/, engine/, alerts/) در کنار فایل اصلی SmartEnergySystem.py قرار دارند.
- **اجرای سیستم:** ترمینال را در پوشه ریشه پروژه باز کرده و دستور زیر را اجرا کنید:

```
• Bash
• python SmartEnergySystem.py
```

- **مشاهده خروجی:** سیستم شروع به پایش کرده و گزارش‌های لحظه‌ای خود را در فایل system\_log.txt ذخیره می‌کند.

### ۳- نحوه اجرای تست‌ها (Testing)

برای اطمینان از صحت عملکرد ماژول‌ها پیش از استقرار نهایی، از اسکریپت تست استفاده کنید:

```
• Bash
• python tests/test_sensor.py
```

این اسکریپت سلامت کلاس Temperature Sensor را در شرایط مختلف بررسی کرده و گزارش می‌دهد.

#### ۴- ملاحظات محیطی (Environment Considerations)

- مدیریت فایل‌ها: سیستم به صورت خودکار فایل‌های مورد نیاز خود را ایجاد می‌کند. فایل `.gitignore`. موجود در پروژه، تضمین می‌کند که فایل‌های غیرضروری (مانند `--pycache--` یا لاگ‌های قدیمی) وارد مخزن گیت نشوند.
- توسعه آتی: در صورت نیاز به افزودن کتابخانه‌های جدید، فایل `requirements.txt` در ریشه پروژه قرار دارد که می‌توانید کتابخانه‌های مورد نیاز را در آن لیست کرده و با دستور `pip install -r requirements.txt` نصب کنید.

#### تطابق کد با الگوهای طراحی (Design Patterns)

معماری سیستم مدیریت انرژی هوشمند بر اساس اصول طراحی شیء‌گرا و با بهره‌گیری از چندین الگوی طراحی استاندارد برای افزایش پایداری و توسعه‌پذیری پیاده‌سازی شده است:

#### ۱- الگوی تزریق وابستگی (Dependency Injection)

این الگو در کلاس‌های `RuleEngine` و `SystemManager` به کار گرفته شده است. به جای اینکه ماژول‌ها، وابستگی‌های خود (مانند سنسور یا لاگر) را در داخل خود ایجاد کنند، این وابستگی‌ها از طریق متد سازنده (`--init--`) به آن‌ها تزریق می‌شود.

#### ۲- الگوی طراحی نما (Facade Pattern)

کلاس `SystemManager` در سیستم ما نقش `Facade` را ایفا می‌کند. این کلاس یک رابط ساده و متمرکز برای تعامل با زیرسیستم‌های پیچیده (موتور قوانین، سنسورها، لاگر) فراهم کرده است. کاربر نهایی فقط کافیس متد `run_cycle` را صدا بزند، بدون اینکه درگیر جزئیات پیچیده‌ی کارکرد سایر ماژول‌ها شود.

#### ۳- الگوی استراتژی پیکربندی (Configuration Strategy)

استفاده از کلاس `Configuration` باعث شده است که منطق تصمیم‌گیری سیستم (موجود در `RuleEngine`) از پارامترهای عملیاتی (دمای هدف) جدا شود. این جداسازی، «تغییرات محیطی» را بدون تغییر در «کد اصلی» ممکن می‌سازد.

#### ۴- اصل مسئولیت واحد (Single Responsibility Principle - SRP)

تمام ماژول‌های پروژه (از جمله Logge ، AlertManager و Report) تنها دارای یک وظیفه‌ی مشخص هستند. این موضوع باعث شده است که کد پروژه بسیار خوانا باشد و نگهداری آن در طول زمان به سادگی صورت پذیرد.